SE4095 Social Network Theory and Practice - Assignment 03

Budgeted Seed Selection on a Workplace Contact Network with REINFORCE (MLP and Transformer Policies)

Mahad Bashir | Roll No. 23F3011

The task is to choose five participants who receive an awareness message directly, so that the message spreads as far as possible in the workplace contact network. An agent picks the five users one by one, and two neural policies (an MLP and a Transformer) are trained with REINFORCE. They are compared with Random and Degree selection on the same 100 diffusion realizations. Everything is produced by python main.py --output results, and the numbers in this report come from the files in results/.

1. Dataset and Network Analysis

The data is the SocioPatterns "Contacts in a Workplace" dataset (office building in France, 24 June to 3 July 2013), in the static package given by the instructor (nodes.csv, edges.csv, departments.csv). I checked the SHA-256 hashes against dataset_README.md and used the files without changes. All 92 nodes are added first and then the 755 edges, so the graph is undirected and unweighted with node IDs 0-91. Contact time and duration are not used. Department labels are only used for the figures and the final analysis.

Measure	Value
Participants / edges	92 / 755
Density	0.1804
Connected components	1 (largest = 92, isolated = 0)
Degree (min / median / mean / max)	4 / 15.5 / 16.41 / 44
Degree centrality = degree/(N-1) (mean / max)	0.1804 / 0.4835
Local clustering (min / median / mean / max)	0.167 / 0.404 / 0.426 / 0.782
Department sizes	DSE 34, DMCT 26, DISQ 15, SRH 13, SFLE 4

Table 1. Basic statistics of the workplace contact graph (results/analysis.json).

The graph is quite dense for a social network: about 18% of all possible pairs had at least one contact, the average participant met 16.4 others and the whole graph is one connected component. Figure 1 shows that the departments form visible groups (DSE and DMCT are the largest and sit in the centre, DISQ and SRH are on the sides). The top hubs in Table 2 (84, 58, 14, 12, 27) all belong to DMCT or DSE. Hubs have lower clustering (0.25-0.40) than the average (0.426), because their contacts come from different groups that do not all know each other. Many low-degree nodes on the edge of a department have high clustering (up to 0.78), meaning their few contacts are tightly connected among themselves.

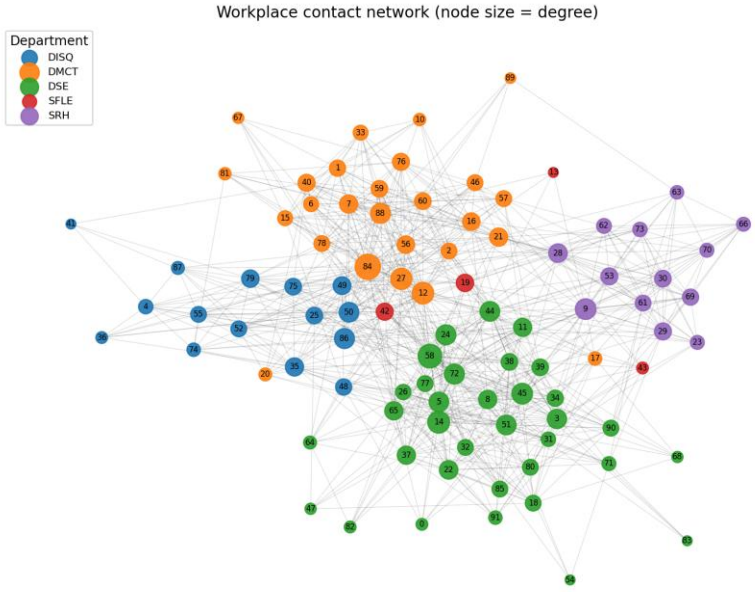


Figure 1. Workplace contact network, coloured by department, node size proportional to degree (results/figures/network.png).

Rank	Node	Degree	Degree centrality	Clustering	Department
1	84	44	0.4835	0.256	DMCT
2	58	38	0.4176	0.283	DSE
3	14	32	0.3516	0.343	DSE
4	12	31	0.3407	0.305	DMCT
5	27	30	0.3297	0.290	DMCT
6	9	28	0.3077	0.251	SRH
7	45	28	0.3077	0.336	DSE
8	24	26	0.2857	0.271	DSE
9	72	26	0.2857	0.403	DSE
10	86	26	0.2857	0.323	DISQ

Table 2. Top-10 participants by degree centrality (ties broken by smaller node ID).

2. Reinforcement Learning Formulation

The environment (environment.py, class SeedSelectionEnv) has reset(diffusion_seed) and step(action), with N = 92 and a budget of 5.

- **Observation:** an N x 5 float matrix in node-ID order with: degree centrality, local clustering coefficient, selected indicator (1 if already chosen), selected-neighbour fraction (selected neighbours / degree, 0 for an isolated node) and remaining budget fraction (remaining selections / 5). The first two are computed once in the constructor; the last three are recomputed after every action. Together with it a Boolean mask is returned that is True for participants not yet selected. No department, original ID or simulator information is in the observation.
- **Action:** an integer 0-91 for an unselected participant. step() raises ValueError for an out-of-range or already selected node and RuntimeError if the episode is already finished; nothing is silently replaced. The policies set the logits of masked participants to -inf before the softmax, so their probability is exactly 0.
- **Reward and termination:** the first four rewards are 0. After the fifth selection the cascade is run once and the reward is R = final spread / N, which is between 0 and 1. The episode terminates right after action five. There are no penalties or shaping terms.

This is a sequential decision problem and not five separate choices. Each selection removes one option from the mask, flips the selected indicator of that user and changes the selected-neighbour fraction of all its neighbours, so the state seen at step t+1 depends on the choices made before. The remaining budget fraction tells the policy how many picks are left. The final reward depends on the combination of the five users: five popular users that know the same people reach overlapping audiences, so a good set can contain users that are individually less popular but cover different parts of the network. Training and evaluation use the same graph; only the diffusion realization (diffusion seed) changes between episodes.

3. Independent Cascade Simulator

independent_cascade() in environment.py follows the required live-arc procedure. Every undirected edge is turned into two directed arcs and the 1,510 arcs are sorted by (source, target) once (make_arcs). For a simulation, numpy.random.default_rng(diffusion_seed) draws one uniform number per arc in this fixed order, and an arc is live if its number is below p = 0.10. A breadth-first search from the distinct seeds over live arcs gives the active set, and the final spread is its size (seeds included). This is the same as the usual cascade description (each newly active user gets one chance to activate each inactive neighbour with probability 0.10) because every arc's coin is flipped only once. The random numbers and the live arcs are local variables inside the function; the policy only gets the feature matrix and the mask, and the simulation is run only after the fifth action, so a policy cannot look at hidden arcs or try hypothetical actions. p = 0 and p = 1 are only used in the checks; all training and evaluation use 0.10.

Check (results/checks.json)	Result
Empty seed set gives spread 0	pass
p = 0 with seeds [3, 3, 10, 50] gives 3 (only distinct seeds)	pass
p = 1 activates every node reachable from the seeds	pass
Same graph, seeds and diffusion seed give same spread (50 seeds)	pass
Selecting an already selected node raises ValueError	pass
Masked nodes get probability 0 (both policies), probabilities sum to 1	pass
All 800 evaluation episodes have 5 distinct users	pass
Reward = spread / 92 in all training and evaluation episodes	pass
All 300 losses and gradient norms are finite	pass

Table 3. Verification checks recorded in checks.json.

4. REINFORCE Implementation

REINFORCE (Williams, 1992) is written by hand in reinforce.py with PyTorch. During training, actions are sampled from the masked categorical distribution and the log-probability of every chosen action is kept (with its computation graph) until the update. After 20 complete episodes the loss is

$$L(\theta) = -(1/20)\,\Sigma_{i=1}{}^{20}\,(R_i - b)\,\Sigma_{t=1}{}^{5}\log \pi\theta(a_{i,t} \mid s_{i,t})$$

Because the discount factor is 1 and only the last reward is non-zero, the return of every step in episode i is the same R_i. The rewards are put in a plain tensor and b is a Python float, so neither receives gradients. Then one Adam step is taken (learning rate 0.001) after clipping the gradient norm to 1.0. The baseline starts at b = 0 in every run, stays fixed while a batch is collected and its loss computed, and is updated only after the optimizer step with $b \leftarrow 0.9b + 0.1\bar{R}$, where $\bar{R}$ is the mean reward of the batch. The trajectories are then discarded. Each run has 1,000 episodes = 50 updates, with training seeds 7, 21 and 42. At the start of a run the Python, NumPy and PyTorch seeds are set to the training seed (before the network is created), and episode e uses diffusion_seed = 1000000 + 10000s + e. No critic, PPO, entropy bonus, normalization, schedule or early stopping is used.

How the update changes behaviour: the gradient of L is $-(R_i - b)\nabla \log \pi$. If an episode got more than the baseline ($R_i > b$), the step increases the log-probability of all five actions taken in that episode in those states; if it got less, their probability is decreased. Over many batches, users (and, because the weights are shared across participants, users with similar features) that appear in better-than-usual seed sets get higher logits. Why the baseline helps: since $E[\nabla \log \pi] = 0$, subtracting a constant b does not change the expected gradient, but it changes the variance. Without a baseline every reward is positive (about 0.65 here), so every sampled action is pushed up and only small differences between the large weights carry information. With b close to the average reward, the weights become small positive or negative numbers, which gives less noisy updates. Note that with b starting at 0 and a factor of 0.1, b reaches about $(1 - 0.9^k)$ of the average reward after k updates, so the first ~10 updates act almost like having no baseline.

5. Policy Architectures

Both policies (models.py) take the same N x 5 features and output one logit per participant; the same masked_distribution() function is used for both, and the same train() function trains both. REINFORCE is the learning algorithm and the MLP/Transformer are only two different policy architectures (functions from the state to logits).

MLP policy: the same network 5 → Linear(5,64) → ReLU → Linear(64,64) → ReLU → Linear(64,1) is applied to each participant's row separately (4,609 parameters, no dropout). The score of a participant only depends on its own five features; it can only know about the other chosen users through its own selected-neighbour fraction and the remaining budget.

Transformer policy: every participant is one token. The 5 features are projected to 64 dimensions with Linear(5,64), passed through one nn.TransformerEncoderLayer (d_model 64, 4 heads, feed-forward 128, ReLU, dropout 0, post layer normalization, default biases, batch_first) and Linear(64,1) gives the logit of each token (33,921 parameters). There is no positional or node-ID embedding, no causal mask, no attention mask and no extra final normalization.

- **Queries, keys and values:** for every token x_i the layer computes a query $q_i = W_Q\, x_i$, a key $k_i = W_K\, x_i$ and a value $v_i = W_V\, x_i$. Token i gives weight $\text{softmax}(q_i \cdot k_j / \sqrt{d})$ to token j and its output is the weighted sum of the values v_j, so each participant collects information from the participants that are relevant for it.
- **Multi-head attention:** the 64 dimensions are split into 4 heads of 16. Each head has its own Q/K/V projections, so different heads can look at different things (for example one at selected users and one at hubs). The head outputs are concatenated and mixed with an output projection.
- **Residual connections and normalization:** with post-normalization the layer computes h = LayerNorm(x + MultiHeadAttention(x)) and then out = LayerNorm(h + FFN(h)) with FFN = Linear(64,128) → ReLU → Linear(128,64). The residual connection keeps the token's own information and makes gradients flow easily; layer normalization keeps the scale of each token vector stable.
- **Attention vs. graph connectivity:** the graph edges are the fixed observed contacts, while attention is a learned, dense weighting over all 92 tokens that depends on the features. Two users that never met can still attend to each other, and neighbours are not treated specially. The graph structure only enters through the features (degree, clustering, selected-neighbour fraction). Since there are no positional embeddings, the model is permutation-equivariant: it treats the participants as a set.
- **Effect of selected users on later choices:** selected participants stay in the attention (they are only masked at the final softmax). Their selected indicator is 1, so their keys and values change after each pick, and the context every other token receives changes too. The Transformer can therefore learn, for example, to lower the score of users that look like already chosen ones, which the MLP cannot do directly.

6. Results

Evaluation uses diffusion seeds 900000-900099 for all 8 rows (800 episodes). Degree takes the five highest-degree users (ties by smaller ID). Random uses default_rng(diffusion_seed + 10000000) and samples five users without replacement. The neural policies are loaded from their final checkpoints and pick the highest-probability valid action at every step (torch.argmax returns the smaller ID on exact ties) without gradients. Selection time is only the time spent choosing the five users; the diffusion is excluded.

Method	Train seed	Spread mean	Spread std	Reward mean	Selection (ms)	Training (s)
random	-1	60.24	9.72	0.6548	0.071	0.00
degree	-1	59.67	7.26	0.6486	0.043	0.00
reinforce_mlp	7	58.88	13.72	0.6400	1.605	4.50
reinforce_mlp	21	58.09	16.11	0.6314	1.459	4.32
reinforce_mlp	42	55.62	18.37	0.6046	1.370	4.37
reinforce_transformer	7	58.09	16.11	0.6314	3.135	10.58
reinforce_transformer	21	59.67	7.26	0.6486	3.078	10.42
reinforce_transformer	42	58.81	14.74	0.6392	2.948	10.47

Table 4. summary.csv: 100 evaluation realizations per row, population standard deviation.

Architecture level (mean and population std of the three run-level mean spreads): REINFORCE MLP 57.53 ± 1.39, REINFORCE Transformer 58.86 ± 0.65. For comparison, Random gives 60.24 and Degree 59.67.

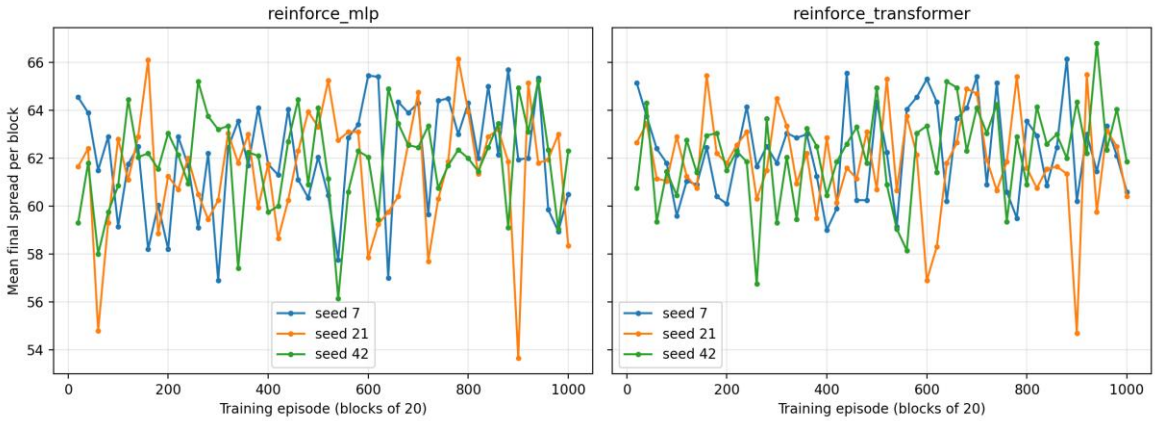


Figure 2. Learning curves: mean final spread of each non-overlapping block of 20 training episodes (50 blocks per run).

Method	Seed	Selected users (order)	Departments	Degree sum	1-hop reach	Edges among seeds	Dept. coverage
degree	-1	84, 58, 14, 12, 27	DMCT, DSE, DSE, DMCT, DMCT	175	75	8	2
mlp	7	66, 0, 36, 82, 85	SRH, DSE, DISQ, DSE, DSE	47	44	0	3
mlp	21	0, 36, 66, 82, 10	DSE, DISQ, SRH, DSE, DMCT	40	43	0	4
mlp	42	13, 71, 26, 89, 17	SFLE, DSE, DSE, DMCT, DMCT	44	36	1	3
transformer	7	66, 0, 36, 82, 10	SRH, DSE, DISQ, DSE, DMCT	40	43	0	4
transformer	21	84, 58, 14, 12, 27	DMCT, DSE, DSE, DMCT, DMCT	175	75	8	2
transformer	42	0, 36, 66, 82, 64	DSE, DISQ, SRH, DSE, DSE	41	38	1	3
random	-1	different every episode	-	-	-	-	3.09 (mean)

Table 5. Selected users and department coverage. Degree and every learned checkpoint choose the same set in all 100 episodes. 1-hop reach = seeds plus their direct neighbours.

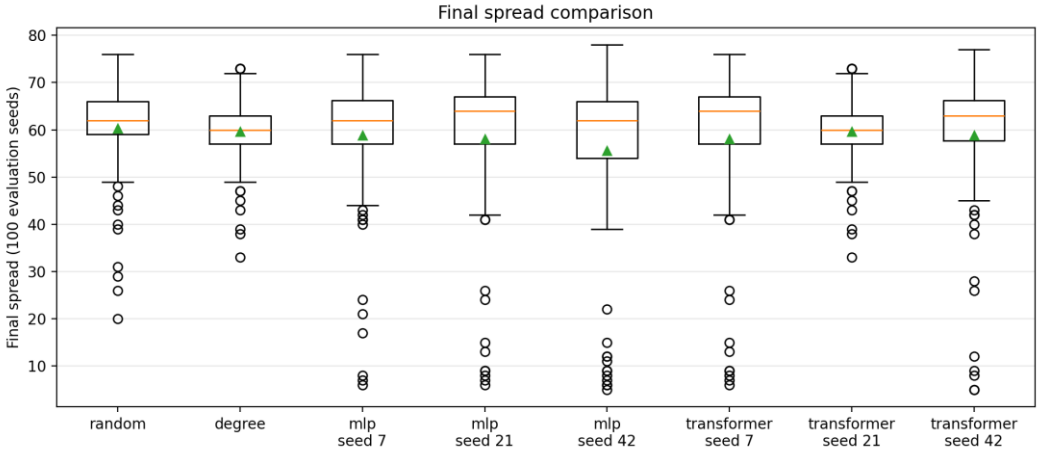


Figure 3. Final spread over the 100 evaluation realizations (box = quartiles, line = median, triangle = mean).

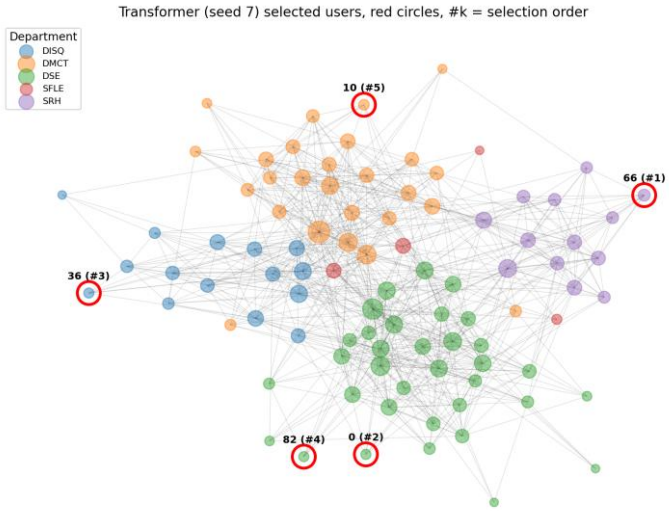


Figure 4. Users selected by the Transformer checkpoint of training seed 7 (#k = selection order).

Discussion

Overall result: no method is clearly better. Random has the highest mean (60.24), Degree is close (59.67) with the lowest std (7.26), and the learned runs are between 55.62 and 59.67. These differences of 1-4 users are small compared to the std of 7-18 users inside each row, so with 100 realizations they should not be over-read. The reason is the diffusion regime: with mean degree 16.4 and p = 0.10 each user has on average 1.64 live out-arcs, which is above 1, so in most realizations the live arcs form one large connected part of about 60-65 users. Any seed set that touches this part reaches roughly the same users, so the final spread is decided mostly by the realization and only a little by the seed set.

Overlapping audiences: the five hubs have a degree sum of 175, but together with their neighbours they reach only 75 distinct users, and 8 of the 10 possible pairs between them are directly connected. They almost always hit the large live component (lowest outcome 33, lowest std), but they reach the same people, so the extra four hubs add almost nothing. Random seeds are spread out (coverage 3.09 departments on average) and sometimes also start small separate cascades, which is probably why its mean is a bit higher.

Learned selections: four of the six runs (MLP 7, MLP 21, Transformer 7, Transformer 42) ended up choosing low-degree, high-clustering users such as 0, 36, 66 and 82 (degree 7-11, clustering 0.67-0.78) at the edges of different departments (Figure 4). These sets have no overlap (0-1 edges among the seeds) and the best department coverage (3-4), but because each seed has few contacts, in some realizations none of them has a live arc into the big component. Then the spread drops below 30 users, sometimes to only 5 (the low outliers in Figure 3), which gives the large std (13.7-18.4) and a lower mean. Transformer seed 21 converged to exactly the Degree set and therefore has identical results; MLP seed 21 and Transformer seed 7 found the same five users in a different order. With greedy evaluation the logit is a smooth function of mainly the two static features, so the argmax lands on the extreme corners of the (degree, clustering) feature space, and a small change in the learned weights gives a completely different seed set. Nodes 0 and 36 even have identical features, and the tie goes to the smaller ID.

Learning stability: the learning curves (Figure 2) are flat and noisy: block means move between about 54 and 67 for all six runs without a clear upward trend. The effect of the seed set on the reward is about 1 user (0.01 reward), while the noise of a single realization is about 9 users (0.1 reward), and there are only 50 updates with learning rate 0.001. In addition, b starts at 0, so in the first updates every sampled action gets a positive weight and the policy can drift towards whatever it sampled early. This is why the final policies differ so much between training seeds. The Transformer runs had a smaller spread of run means (std 0.65 vs 1.39), but with three seeds this is weak evidence.

Computational cost: one MLP run took about 4.3-4.5 s and one Transformer run 10.4-10.6 s on a laptop CPU, including the training simulations. Choosing five users took about 1.48 ms for the MLP and 3.05 ms for the Transformer (self-attention over all 92 tokens at each of the 5 steps), compared to 0.043 ms for Degree and 0.071 ms for Random. Degree needs no training at all and was the most stable method on this graph.

Department coverage: Degree covers only 2 departments (DMCT and DSE), Random 3.09 on average and the learned sets 2-4. Coverage is only descriptive and was never used for training or tuning. Here a higher coverage did not give a higher spread, because the users that cover several departments were the low-degree ones.

7. Limitations

- **One static graph:** all results are for this one aggregated 92-person graph. Aggregation removes contact timing, frequency and duration, so two people who met once count the same as two people who met every day.
- **Simulated spread:** the spread comes from an Independent Cascade simulation with an assumed p = 0.10; it is not observed message adoption. With a smaller p the seed choice would probably matter more.
- **Small training budget:** only three training seeds, 1,000 episodes (50 updates) and 100 evaluation realizations were allowed, and no hyperparameter search was done. Given the noisy reward, this is not enough for the policies to learn a clear strategy, so the results say more about the setup than about the best possible RL agent.
- **Features and timing:** the policies only see five simple features, the evaluation is greedy (deterministic) and timings were measured in Python on one laptop, so they include Python overhead and change slightly between runs.

References

[1] M. Génois, C. L. Vestergaard, J. Fournet, A. Panisson, I. Bonmarin, A. Barrat. Data on face-to-face contacts in an office building suggest a low-cost vaccination strategy based on community linkers. Network Science 3(3), 326-347, 2015. https://doi.org/10.1017/nws.2015.10

[2] SocioPatterns. Contacts in a workplace (2013 release, CC Public Domain Dedication). https://sociopatterns.org/datasets/contacts-in-a-workplace/

[3] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229-256, 1992.

[4] D. Kempe, J. Kleinberg, É. Tardos. Maximizing the spread of influence through a social network. KDD 2003.

[5] A. Vaswani et al. Attention is all you need. NeurIPS 2017.

[6] PyTorch, NetworkX and NumPy documentation (nn.TransformerEncoderLayer, distributions.Categorical, nn.utils.clip_grad_norm_, networkx.clustering, numpy.random.default_rng). No external code was copied; the libraries were used as described in their documentation.